

**Final Project Report
BSCS-2nd Semester
Spring 2026**

Subject: Object Oriented Programming Lab



Submitted to:

Sir Mohsin Shahzad

Submitted By:

Name	Zainab Amir Laiba Javed
Reg No	25-CS-104 25-CS-028

**Department of CS
HITEC University, Taxila**

Project Name :

School Campus Management System

GitHub Respositry :

[https://github.com/zainab6491245-arch/\[HITEC-OOP-SCMS-104\]](https://github.com/zainab6491245-arch/[HITEC-OOP-SCMS-104])

1. Introduction :

Modern university campuses manage a wide range of interconnected operations: enrolling and tracking students, assigning faculty to courses, cataloging library resources, collecting fees, allocating hostel rooms, and producing administrative reports. Traditionally these tasks are handled by separate, loosely-related tools, making the overall workflow inconsistent and difficult to maintain. The Smart Campus Management System (SCMS) addresses this by providing a single, fully object-oriented C++ application that models every one of these operations as a cooperating set of classes. The primary goal of this project was not simply to produce a working program, but to deliberately apply every major Object-Oriented Programming concept covered in the CS-104L course — encapsulation, inheritance (single, multi-level and multiple), polymorphism, abstraction, operator overloading, static members, deep-copy semantics, exception handling, and file persistence — inside one coherent system. Each of the six modules described in this report was designed around a specific subset of these concepts, so that the final codebase demonstrates practical, working examples of all 25 required OOP concepts rather than isolated textbook exercises. A secondary goal was to practice a professional software development workflow: version-controlling the project on GitHub from day one, committing incrementally by phase, and documenting the design with a UML class diagram and this report .

2. System Design :

The system is organized into six modules, each centered on a small group of related classes. The UML class diagram below shows every class implemented in the project, grouped and color-coded by module, along with the relationships between them .

3. OOP Concepts Implementation :

The table below maps every one of the 25 mandatory OOP concepts to the exact class(es) where it is implemented, with a short explanation and a representative code snippet from the actual project source.

Classes & Objects ,Encapsulation ,Pure Virtual,Constructor,Destructor (Person.h +.cpp) :

Every module is built from C++ classes that bundle related data and behaviour together, e.g. Student, Course, Book and Invoice, instantiated as objects in main.cpp. All data members are private or protected. Outside code can only read or modify them through public getter functions, preventing uncontrolled access.

```
1. #ifndef PERSON_H // aksar baray projects m bohat si files hoty ha to unko
   include karna parta ha aur agar kisi file ko do bar include kar diya to error
   a jata ha isliye hum include guard use karte ha taki agar file do bar include
   ho jaye to compiler usko ignore kar de
2. #define PERSON_H // ye line include guard ka start hai, agar PERSON_H define
   nahi hai to usko define kar do aur is file ko include kar do, agar PERSON_H
   already defined hai to is file ko ignore kar do
3. #include <string>
4. #include <iostream>
5. using namespace std;
6.
7. class Person
8. {
9. protected:
10.     string name;
11.     string cnic;
12.     int age;
13.     string contact;
14.
15. public:
16.     Person(string n,string c,int a,string cont);
17.     virtual void displayInfo()
18.         const = 0;
19. virtual ~Person() = default;
20.     string getName() const;
21.     string getCnic() const;
22.     int getAge() const;
23.     string getContact() const;
24.
25. };
26. #endif // ab hi tak humne person class banayi hai jisme humne name, cnic, age
    aur contact ko protected banaya hai taki derived classes unko access kar sake
    aur humne displayInfo function ko pure virtual function banaya hai taki
```

derived classes usko override kar sake aur apni information display kar sake, humne destructor ko virtual banaya hai taki agar kisi derived class ka object delete kiya jaye to uska destructor bhi call ho jaye, humne getter functions bhi banaye hai taki hum name, cnic, age aur contact ko access kar sake

Single Inheritance(Student.h +.cpp) :

Student inherits directly and only from the abstract Person base class, reusing its protected attributes and overriding displayInfo().

```
#ifndef STUDENT_H
#define STUDENT_H

#include "person.h" //is ka matlab hai ke student class person class se inherit
kar rahi hai, isliye hume person.h file ko include karna parta hai taki student
class person class ke members ko access kar sake

class Student : public Person
{
private:
    int studentrollno;
    double gpa;
    int semester;
    string enrolledcourses[10]; // Assuming a maximum of 10 courses
    int numCourses; // To keep track of the number of enrolled courses

public:
    Student(string n, string c, int a, string cont, int id, double grade, int
sem, string courses[], int num);
    void displayInfo() const override;
    int getStudentrollno() const;
    double getGpa() const;
    int getSemester() const;
    char calculateGrade() const; // ye function GPA ke basis par grade calculate
karega, agar GPA 3.5 se zyada hai to grade A hoga, agar GPA 3.0 se zyada hai to
grade B hoga, agar GPA 2.5 se zyada hai to grade C hoga, agar GPA 2.0 se zyada
hai to grade D hoga aur agar GPA 2.0 se kam hai to grade F hoga
    void enrollCourse(string course);
};

#endif // STUDENT_H
```

Multilevel Inheritance (Graduate Student.h):

GradStudent inherits from Student, which itself inherits from Person, forming a three-level chain.

```
#ifndef GRADSTUDENT_H
#define GRADSTUDENT_H
#include "student.h"
class GradStudent : public Student
{
private:
    string ThesisTopic;
    string AdvisorName;
public:
    GradStudent(string n, string c, int a, string cont, int id, double gpa, int
sem, string topic, string advisor);
    void displayInfo() const override;
    string getThesisTopic() const;
    string getAdvisorName() const;
};
#endif // GRADSTUDENT_H
```

Operator Overloading, Friend Functions, Aggregation (Course .h + .cpp) :

Course overloads == to compare two courses by courseCode and << to print course details; The stream-insertion operator for Course is declared as a friend so it can access private members while remaining a free function usable with cout.

```
#ifndef COURSE_H
#define COURSE_H
#include <string>
#include <iostream>
using namespace std;

class Faculty;
class Student;

class Course
{
private:
    string code;           // course ka code, jaise "CS-104L"
    string title;          // course ka naam, jaise "OOP"
    int creditHours;       // kitne credit hours hain, jaise 3
    Faculty* teacher;      // konsa Faculty ye course parhata hai
    int seatLimit;         // max kitne students enroll ho sakte hain
    int seatsFilled;       // abhi kitne students enroll ho gaye
};
```

```

Student* students[100]; // enrolled students ki list
Student* waitingList[10]; // waiting list ke liye array (max 10 students)
int waitingCount; // waiting list mein kitne students hain

public:
    Course(string c, string t, int hours, Faculty* t1, int limit);
    void setTeacher(Faculty* t1);
    void enroll(Student* s);
    void displayInfo() const;

    string getCode() const;
    string getTitle() const;
    int getCreditHours() const;
    int getWaitingCount() const;
    int getSeatsFilled() const;
    int getSeatLimit() const;
    Student* getWaitingStudent(int index) const;

    bool operator==(const Course& other) const;
    Student** operator+(const Course& other);
    friend ostream& operator<<(ostream& out, const Course& c);
};

#endif // COURSE_H

```

Copy Constructor, Operator-= (FeeRecord.h +.cpp) :

FeeRecord overloads -= to record a payment and reduce the balance. FeeRecord defines an explicit copy constructor that copies every field value-by-value into the new object, guaranteeing the copy is fully independent.

```

#ifndef FEERECORD_H
#define FEERECORD_H
#include<string>
#include<iostream>
using namespace std;
class FeeRecord
{
private:
    string studentRef;
    double semesterFee;
    double hostelFee;
    double libraryFine;
    double totalpaid;
    double balance;
public :

```

```

    FeeRecord(string Ref, double semFee, double hFee, double libFine);
    FeeRecord(const FeeRecord& other);
    FeeRecord& operator=(const FeeRecord& other);
    ~FeeRecord();
    FeeRecord& operator--(double amount) ;// Overloading the '-' operator to
subtract fee records
    void displayInfo() const;
    string getStudentRef() const;
    double getSemesterFee() const;
    double getHostelFee() const;
    double getLibraryFine() const;
    double getBalance() const;
};

#endif // !FEERECORD

```

Static Members (Invoice.h +.cpp) :

Invoice::invoiceCounter is a static int shared by every Invoice object; each new invoice increments the same counter to generate an auto-incrementing unique ID.

```

#ifndef INVOICE_H
#define INVOICE_H
#include <string>
#include <iostream>
using namespace std;

class Invoice
{
private:
    int invoiceID;
    string date;
    string* items;
    int itemCount;
    double totalAmount;
    static int invoiceCounter;

public:
    Invoice(string date, string* items, int itemCount, double totalAmount);
    Invoice(const Invoice& other);
    ~Invoice();
    static int getInvoiceCounter();
    int getInvoiceID() const;
    string getDate() const;

```

```

    double getTotalAmount() const;
    void displayInfo() const;
};
#endif

```

Multiple Inheritance (HostelManager.h+.cpp):

HostelManager inherits simultaneously from two unrelated abstract interfaces, Accommodation and Reportable, combining both behaviours in one class.

```

#ifndef HOSTELMANAGER_H
#define HOSTELMANAGER_H

#include "accomodation.h"
#include "reportable.h"
#include "hostelblock.h"
#include <string>
#include <iostream>
using namespace std;
class HostelManager : public Accomodation, public Reportable //multiple
inheritance
{
private:
    HostelBlock* block;    // composition – pointer se reference rakha hai ,that's
allow to make memory dynamicalyand cleanup through destructor

public:
    HostelManager(string blockName);
    ~HostelManager();

    void allocateRoom() override;
    void vacateRoom() override;
    void generateReport() const override;

    HostelBlock* getBlock() const;
};

#endif // HOSTELMANAGER_H

```

Composition(Hostel.h+.cpp):

```

// Empty block for composition code

```

```

#ifndef HOSTELBLOCK_H
#define HOSTELBLOCK_H

#include "room.h"
#include <string>
#include <iostream>
using namespace std;
class HostelBlock
{
private:
    string blockName;
    Room rooms[10];    // array of Room OBJECTS (pointer nahi!) ,if hostel
destroy rooms automatically destroy
    int roomCount;

public:
    HostelBlock(string name);

    void addRoom(Room room);
    void displayAllRooms() const;

    string getBlockName() const;
    int getRoomCount() const;
};

#endif // HOSTELBLOCK_H

```

Exception Handling (Exception .h):

CapacityExceededException is thrown when Course::enroll() exceeds maxCapacity, and OverdueException models an overdue-fine scenario; both are caught in try/catch blocks in main().

```

#ifndef EXCEPTION_H
#define EXCEPTION_H
#include <exception>
using namespace std;

class CapacityExceededException : public exception
{
public:
    const char* what() const noexcept override
    {
        return "Course capacity exceeded. Student added to waiting list.";
    }
}

```

```
};
class OverdueException : public exception
{
public:
    const char* what() const noexcept override
    {
        return "Book return is overdue. Please return the book as soon as
possible.";
    }
};
#endif // EXCEPTION_H
```

File I/O (library.h):

Library::saveToFile() and loadFromFile() use ofstream/ifstream to persist the catalog as pipe-delimited text, tagging each line BOOK or JOURNAL so it can be reconstructed correctly.

```
#ifndef LIBRARY_H
#define LIBRARY_H
#include "libraryitem.h"
#include<fstream>
using namespace std;
class Library
{private :
    LibraryItem* items[50];
    int itemCount;
public:
    Library();
    void addItem(LibraryItem* item);
    void displayAllItems() const;
    void saveToFile(string filename) const;
    void loadFromFile(string filename);
    LibraryItem* searchByTitle(string searchTitle) const;
    ~Library();
};
#endif // LIBRARY_H
```

Sorting/Searching, std::sort, std::find if(Reports .h+cpp):

Reports.h / Reports.cpp provide standalone utility functions — sorting, searching and a consolidated campus report — that operate on arrays of Student pointers independently of any single class. Reports::sortStudentsByGPA() uses the STL std::sort algorithm with a lambda comparator to order students by descending GPA.

```
#ifndef REPORTS_H
#define REPORTS_H

#include "../person/Student.h"
#include <string>
#include <iostream>
#include <algorithm> //std::sort + std::find yahan sa ata ha
using namespace std;

void sortStudentsByGPA(Student* students[], int count); //student ki array gpa ka
hisaab sa sort krnay ka liay by using std::sort
void displayStudents(Student* students[], int count); //print -> sorted/unordered
Student* findStudentByRollNo(Student* students[], int count, int rollNo); //use
std::find to find specific rollno
void generateCampusReport(Student* students[], int studentCount, int totalBooks,
double totalFeeCollected, int totalRoomsOccupied); //generate a summary of report
by combining all

#endif // REPORTS_H
```

4. Module Descriptions :

Module 1 – Person Hierarchy :

Forms the backbone of the system. The abstract Person class defines shared attributes (name, CNIC, age, contact) and a pure virtual displayInfo(). Student, Faculty and Staff derive from Person, and GradStudent further derives from Student to demonstrate multi-level inheritance. The design decision to make displayInfo() pure virtual forces every concrete subclass to provide its own display behaviour, the foundation for runtime polymorphism used throughout the rest of the system .

Module 2 – Course & Enrollment Management :

Course models an offered course with a capacity-limited enroll() method, and Enrollment links a specific Student to a specific Course with a date and grade. operator== lets two Course objects be compared by code, while a friend operator<< prints a course's details. Enrolling beyond maxCapacity throws a custom CapacityExceededException, demonstrating defensive design through exception handling rather than silent failure .

Module 3 – Library System :

An abstract `LibraryItem` base class unifies `Book` and `Journal` under a common interface (`checkout()`, `displayInfo()`). `Library` manages a fixed array of `LibraryItem*` so that both kinds of items can be stored polymorphically. `saveToFile()/loadFromFile()` persist the catalog to a text file, tagging each record by type so it can be reconstructed as the correct subclass on load .

Module 4 – Fee & Finance Management :

`FeeRecord` tracks a student's fees and balance, with an overloaded `-=` operator to record payments. Its explicit copy constructor and copy-assignment operator guarantee that copying a `FeeRecord` never causes two records to accidentally share state. `Invoice` uses a static `invoiceCounter` so every invoice receives a unique, auto-incrementing ID without any class outside `Invoice` needing to manage it.

Module 5 – Hostel Management :

Two small abstract interfaces, `Accommodation` (room allocation) and `Reportable` (occupancy reporting), are combined through multiple inheritance in `HostelManager`. `Room` objects are composed inside `HostelBlock` (an array of real `Room` objects, not pointers), and `HostelBlock` itself is owned by `HostelManager` — a clear composition chain illustrating that destroying the manager destroys everything beneath it .

Module 6 – Reporting & Utilities :

`Reports.h / Reports.cpp` provide free functions that operate across the whole student population: `sortStudentsByGPA()` (`std::sort` with a lambda), `findStudentByRollNo()` (`std::find_if` with a lambda) and `generateCampusReport()`, which consolidates totals from every other module into a single printed summary.

5. GitHub Workflow :

The project was version-controlled on GitHub from the very first day, using Visual Studio's built-in Git integration. The repository was kept public throughout, with a `README.md` and `.gitignore` added in the very first commit. Branching/commit strategy: A single main branch was used. Commits were made phase-by-phase, matching the module structure of the project rather than committing the entire project in one shot:

Add UML class diagram to docs folder	88f64fc	<>
Add README.md with project info, OOP concepts, and modules	36d7e61	<>
[Final] Add Reports module - sorting, searching, consolidated(summary) campus report	07ed890	<>
[Phase 3] Add Hostel Management module - multiple inheritance, composition, reports	73cd146	<>
[Phase 2] Add Finance Module - Fee Record, Invoice, Static members and copy constructors	93b715e	<>
[Phase 2] Add Library Module - Library item, Book, Journal, File I/O, Exceptions	0d48cee	<>
Commits on Jun 19, 2026		
[Phase 1] Add Course & Enrollment-operator overloading, exceptions	53f1ab8	<>
Commits on Jun 18, 2026		
[Phase 1] Add Person Hierarchy - Person, Student, Staff, Faculty, Gradstudent classes	cd1f2a6	<>
[Setup] Initialize project structure and README	7f4c618	<>
Commits on Jun 17, 2026		
Initial commit	5bcc8b9	<>

6. Challenges & Solutions :

Challenge 1: Constructor chaining across multi-level inheritance

When implementing GradStudent : Student : Person, an early version of the GradStudent constructor passed the wrong parameter types to the Student base-class constructor (e.g. a string where an int semester was expected), producing confusing compiler errors far from the actual mistake. Solution: each constructor's base-class initializer list (: Student(n, c, a, cont, id, gpa, sem, ...)) was checked parameter-by-parameter against the parent's declared constructor signature, and unused parameters were removed.

Challenge 2: Case-sensitivity mismatches between class names

C++ class names are case-sensitive, but several files referred to the same class with inconsistent casing (e.g. libraryItem in one file and LibraryItem in another). This produced a cascade of unrelated-looking errors such as "base class undefined" and "is not a class or namespace name" across multiple files. Solution: one canonical spelling (LibraryItem) was chosen, and Find & Replace was used file-by-file (.h and .cpp) until every reference matched exactly.

Challenge 3: Missing function definitions for declared overrides

Several pure virtual functions were correctly declared with override in a header file but their actual implementation was forgotten in the matching .cpp file (e.g. GradStudent::displayInfo()). The project compiled cleanly but failed at the link stage with “unresolved external symbol” errors. Solution: every override declared in a header was traced back to confirm a matching definition existed in its .cpp file, and the missing functions were added.

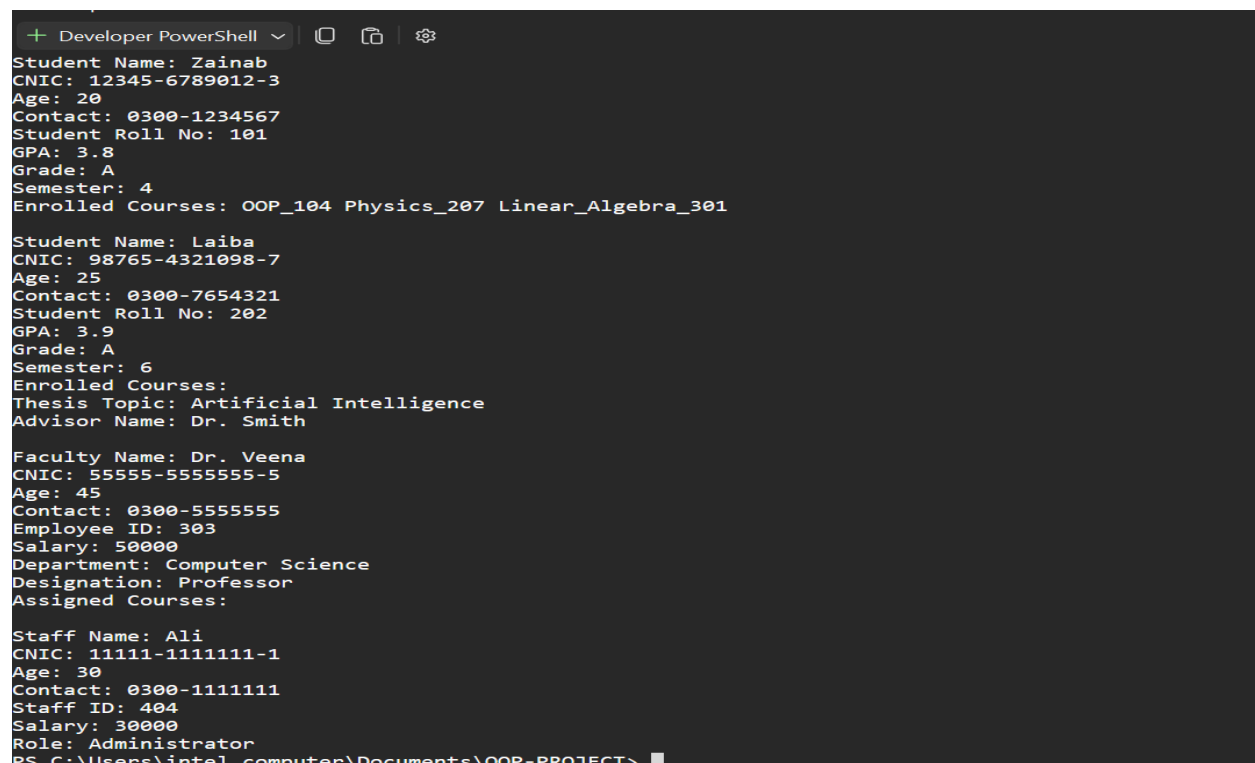
Challenge 4: A single missing semicolon causing dozens of unrelated errors

A header guard typo (#ifnded instead of #ifndef) and a missing semicolon in Library.h caused the compiler to misinterpret the rest of the file, producing over 50 seemingly unrelated errors in main.cpp (“redefinition of cout”, “missing type specifier”, etc.). Solution: rather than trying to fix every reported error individually, only the very first error in the compiler log was treated as the real problem; fixing it resolved the rest automatically .

7. Testing:

Each module was compiled and executed independently as it was completed, using the Microsoft C++ compiler (cl.exe) from the Visual Studio Developer PowerShell, before being integrated into the combined main.cpp. A screenshot of the console output should be inserted below for each module before final submission .

Module 1 – Person Hierarchy



```
+ Developer PowerShell
Student Name: Zainab
CNIC: 12345-6789012-3
Age: 20
Contact: 0300-1234567
Student Roll No: 101
GPA: 3.8
Grade: A
Semester: 4
Enrolled Courses: OOP_104 Physics_207 Linear_Algebra_301

Student Name: Laiba
CNIC: 98765-4321098-7
Age: 25
Contact: 0300-7654321
Student Roll No: 202
GPA: 3.9
Grade: A
Semester: 6
Enrolled Courses:
Thesis Topic: Artificial Intelligence
Advisor Name: Dr. Smith

Faculty Name: Dr. Veena
CNIC: 55555-5555555-5
Age: 45
Contact: 0300-5555555
Employee ID: 303
Salary: 50000
Department: Computer Science
Designation: Professor
Assigned Courses:

Staff Name: Ali
CNIC: 11111-1111111-1
Age: 30
Contact: 0300-1111111
Staff ID: 404
Salary: 30000
Role: Administrator
PS C:\Users\intel_computer\Documents\OOP-PROJECT>
```

Module 2 - Course & Enrollment Management

```
Developer PowerShell
+ Developer PowerShell v | | |
Salary: 30000
Role: Administrator
===== Module 2 : Course and Enrollment =====

Student 1 enrolled successfully in course
Student 2 enrolled successfully in course
Course capacity exceeded. Student added to waiting list.

Course Code: CS_104L
Course Title: Object_Oriented Programming
Credit Hours: 3
Seats Filled: 2/2
Enrolled Students:
===== Enrollment Information =====
Course :
Course Code: CS_104L
Course Title: Object_Oriented Programming
Credit Hours: 3
Seats Filled: 2/2
Student Name:
Student Name: Zainab_Amir
CNIC: 11111-1111111-1
Age: 19
Contact: 0300-7654432
Student Roll No: 104
GPA: 4
Grade: A
Semester: 5
Enrolled Courses: CS_104L
Enrollment Date: 2024-01-15
Grade: 3.8

===== Operator Overloading Demonstration =====
===== Comparing Courses =====
Courses are the same based on course code.
Course Code: CS_104L
Course Title: Object_Oriented Programming
Seats Filled: 2/2
```

Module 3 - Library System

```
Developer PowerShell
+ Developer PowerShell v | | |
Genre: Programming
ISBN: 978-0321563842
Number of Copies: 5

Journal ID: J001
Title: AI Research
Author: John Doe
Publication Year: 2022
ISSN: 1234-5678
Volume: 10
Issue: 2
Field: Artificial Intelligence

Book ID: B002
Title: Data Structures
Author: Mark Allen Weiss
Publication Year: 2014
Genre: Computer Science
ISBN: 978-0132847377
Number of Copies: 3

Searching for Title
Book ID: B001
Title: C++ Programming
Author: Bjarne Stroustrup
Publication Year: 2013
Genre: Programming
ISBN: 978-0321563842
Number of Copies: 5

Checking out a item:
Book checked out successfully.
Journal checked out successfully.
Book checked out successfully.

Saving library data to file...

Demonstrating OverdueException
Book return is overdue. Please return the book as soon as possible.
PS C:\Users\intel computer\Documents\OOP-PROJECT>
```

Module 4 – Fee & Finance Management

```
Developer PowerShell
===== Copy Constructor Demonstration =====
Copy constructor called for student: Zainab-104
Student Reference: Zainab-104
Semester Fee: 50000
Hostel Fee: 20000
Library Fine: 500
Total Paid: 30000
Balance: 40500

===== Copy Assignment Demonstration =====
Assignment operator called for student: Zainab-104
Student Reference: Zainab-104
Semester Fee: 50000
Hostel Fee: 20000
Library Fine: 500
Total Paid: 30000
Balance: 40500

===== Static Member Demonstration (Invoice) =====
Invoice ID: 1
Date: 2024-01-15
Total Amount: 70000
Items: Semester Fee Hostel Fee
Total Invoices Created: 1

Invoice ID: 2
Date: 2024-02-10
Total Amount: 500
Items: Library Fine
Total Invoices Created: 2

Copy Constructor called for Invoice ID: 1
Total Invoices Created: 2
Destructor called for Invoice ID: 1
Destructor called for Invoice ID: 2
Destructor called for Invoice ID: 1
Destructor called for student: Zainab-104
Destructor called for student: Zainab-104
Destructor called for student: Zainab-104
PS C:\Users\intel computer\Documents\OOP-PROJECT>
```

Module 5 – Hostel Management

```
Developer PowerShell
===== Copy Constructor Demonstration =====
Copy constructor called for student: Zainab-104
Student Reference: Zainab-104
Semester Fee: 50000
Hostel Fee: 20000
Library Fine: 500
Total Paid: 30000
Balance: 40500

===== Copy Assignment Demonstration =====
Assignment operator called for student: Zainab-104
Student Reference: Zainab-104
Semester Fee: 50000
Hostel Fee: 20000
Library Fine: 500
Total Paid: 30000
Balance: 40500

===== Static Member Demonstration (Invoice) =====
Invoice ID: 1
Date: 2024-01-15
Total Amount: 70000
Items: Semester Fee Hostel Fee
Total Invoices Created: 1

Invoice ID: 2
Date: 2024-02-10
Total Amount: 500
Items: Library Fine
Total Invoices Created: 2

Copy Constructor called for Invoice ID: 1
Total Invoices Created: 2
Destructor called for Invoice ID: 1
Destructor called for Invoice ID: 2
Destructor called for Invoice ID: 1
Destructor called for student: Zainab-104
Destructor called for student: Zainab-104
Destructor called for student: Zainab-104
PS C:\Users\intel computer\Documents\OOP-PROJECT>
```

Module 6 – Reporting & Utilities

```
Enrolled Courses: OOP_104
Student Name: Zani
CNIC: 11111-1111111-1
Age: 19
Contact: 0300-1111111
Student Roll No: 201
GPA: 3.2
Grade: B
Semester: 3
Enrolled Courses: OOP_104

===== Searching for Roll No 202 =====
Found: Student Name: Sara
CNIC: 22222-2222222-2
Age: 20
Contact: 0300-2222222
Student Roll No: 202
GPA: 3.9
Grade: A
Semester: 4
Enrolled Courses: OOP_104

===== Consolidated Campus Report =====
=====
SMART CAMPUS MANAGEMENT SYSTEM - REPORT
=====
Total Students Enrolled: 3
Total Books in Library: 2
Total Fee Collected: 70500
Total Hostel Rooms Occupied: 1
=====

Destructor called for Invoice ID: 1
Destructor called for Invoice ID: 2
Destructor called for Invoice ID: 1
Destructor called for student: Zainab-104
Destructor called for student: Zainab-104
Destructor called for student: Zainab-104
PS C:\Users\intel_computer\Documents\OOP-PROJECT>
```

8. Conclusion :

Building the Smart Campus Management System turned the individual OOP topics covered in lectures — inheritance, polymorphism, operator overloading, exceptions, static members, file I/O — into a single piece of working software where these concepts had to interact correctly with one another. Designing the Person hierarchy first, then layering Course, Library, Finance, Hostel and Reporting on top of it, made it clear how abstraction and inheritance reduce duplicated code across genuinely different real-world entities. Debugging compiler and linker errors across many small, interconnected files was the most time-consuming part of the project, but it taught a practical lesson that is easy to miss when writing short, single-file programs: a single typo (a header guard, a case mismatch, a missing override definition) can cascade into dozens of unrelated-looking errors, and the fix is almost always the very first error reported, not the last. Maintaining the project on GitHub with phase-based commits also made it possible to track exactly how the design evolved from a bare Person class to a complete six-module system.

9. References :

- Deitel, P. & Deitel, H. C++ How to Program, 10th Edition.
- Stroustrup, B. The C++ Programming Language, 4th Edition.
- Meyers, S. Effective Modern C++.
- cppreference.com – Authoritative C++ standard reference (used for `std::sort`, `std::find_if`, `fstream`).
- learncpp.com – Reference tutorials on inheritance, virtual functions and operator overloading.
- git-scm.com/doc – Official Git documentation.
- docs.github.com – GitHub repository management guides.